

Correction complète des annales Programmation Générique et Programmation Objet (2023 - 2025)

Annale 2023 — Jeu de Dames

Question 1 :

Le damier peut être représenté avec :

```
std::vector<std::vector<char>> damier(8, std::vector<char>(8, ' '));
```

Question 2 :

La classe Dame contient :

- un damier
- le joueur courant
- des méthodes de déplacement

Exemple :

```
class Dame {
private:
    std::vector<std::vector<char>> damier;
    char joueurCourant;

public:
    Dame();
    ~Dame();
    void afficher() const;
    bool deplacerPion(int xD, int yD, int xA, int yA);
};
```

Question 3 :

Le constructeur initialise les pions noirs et blancs.

Question 4 :

Le destructeur est vide car STL gère automatiquement la mémoire.

Question 5 :

Le déplacement vérifie :

- les limites du plateau
- que la case d'arrivée est libre
- le déplacement diagonal

Annale 2023 — Gestion de tables STL

Le meilleur conteneur associatif est :
`std::map<Cle, Valeur>`

Pourquoi ?

- clés uniques
- tri automatique
- accès rapide

Classe Cle :

```
class Cle {
private:
    std::string matricule;

public:
    Cle(std::string m) : matricule(m) {}

    bool operator<(const Cle& c) const {
        return matricule < c.matricule;
    }
};
```

Classe Valeur :

```
class Valeur {
private:
    std::string nom;
    std::string grade;
    std::string telephone;
};
```

```
Ajout :
Table.insert({
    Cle("0079"),
    Valeur("HENRION", "Lieutenant", "2111")
});
```

Annale 2024 — Calcul de fréquences

Structures utilisées :

- `std::vector<std::string>`
- `std::map<std::string, int>`

Classe Livre :

```
class Livre {
private:
    std::vector<std::string> mots;

public:
    Livre(std::string fichier);
    std::vector<std::string> lexique();
    std::map<std::string, int> monogramme();
};
```

Le constructeur lit le fichier texte.

La fonction lexique :

- utilise `std::set`
- retourne les mots uniques

La fonction monogramme :

- calcule les fréquences
- utilise `std::map`

Annale 2024 — Emploi du temps

Classe PlageHoraire :

- jour
- début
- fin
- tâche

Exemple :

```
class PlageHoraire {
private:
    std::string jour;
    int debut;
    int fin;
    std::string tache;
};
```

Le meilleur conteneur pour EmploiTemps :

```
std::vector<PlageHoraire>
```

Pourquoi ?

- taille dynamique
- parcours efficace
- insertion simple

Annale 2025 — Bibliothèque numérique

Le meilleur conteneur :

```
std::vector<Livre>
```

Classe Livre :

- titre
- auteur
- année
- identifiant

Fonctions de Bibliotheque :

- ajouterLivre

- supprimerLivre
- rechercherParAuteur
- afficherParAnnee
- compterLivresApres

Algorithmes STL utilisés :

- std::sort
- std::count_if
- std::remove_if

Annale 2025 — Urgences écologiques

La structure principale :

```
std::priority_queue<UrgenceEcologique>
```

Objectif :

- traiter d'abord les urgences les plus graves
- puis les plus anciennes

Operator < :

```
bool operator<(const UrgenceEcologique& u) const {  
    if(niveau == u.niveau)  
        return date > u.date;  
  
    return niveau < u.niveau;  
}
```

Fonctions :

- ajouterUrgence()
- traiterUrgences()